Developer Quick Start Guide

Van Dyk Recycling Solutions Website - Everything You Need to Know

Welcome! This guide gets you productive in 15 minutes.

What Is This Project?

A **React + TypeScript website** for Van Dyk Recycling Solutions, a company selling recycling equipment and MRF (Material Recovery Facility) technology. The site showcases equipment, solutions, and services with multi-language support.

Live Site: https://stagevdrs.vercel.app
Stack: React 18 + TypeScript + Vite + Tailwind CSS + Vercel Serverless Functions

---

Get Running in 5 Minutes

```bash
# Clone and install
git clone https://github.com/AjithVanDyk/stagevdrs.git
cd staging_vdrs
npm install

# Start development server
npm run dev
# Opens at http://localhost:5173
```

That's it. No environment variables needed for local frontend development.

Project Structure (What's Where)

```
staging_vdrs/
├── public/Images/        # All images (URL-encoded for Vercel)
├── src/
│   ├── components/       # Reusable UI (Navbar, Footer, Modal, etc.)
│   ├── pages/            # 40+ page components
│   ├── config/
```

```
│   │   ├── images.ts      # Centralized image paths
│   │   └── translations.ts # EN/FR/ES translations
│   ├── hooks/             # Custom hooks (useTranslation, etc.)
│   ├── utils/             # Helpers (animations, SEO, etc.)
│   └── App.tsx            # Routes defined here
├── api/                   # Serverless functions (contact, quote, etc.)
└── vercel.json            # Deployment config
```

The Three Things You'll Work On Most

1. Pages (`src/pages/`)

Each page is a React component. Routes are defined in `App.tsx`:

tsx

```tsx
// Adding a new page:
// 1. Create src/pages/MyNewPage.tsx
// 2. Add lazy import in App.tsx:
const MyNewPage = lazy(() => import('./pages/MyNewPage'));
// 3. Add route:
<Route path="/my-new-page" element={<MyNewPage />} />
```

2. Translations (`src/config/translations.ts`)

Three languages: English (en), French (fr), Spanish (es).

tsx

```
// In your component:
import { useTranslation } from '../hooks/useTranslation';

function MyComponent() {
  const { t } = useTranslation();
  return <h1>{t('home.title')}</h1>;
}

// Add to translations.ts:
export const translations = {
  en: { home: { title: 'Welcome' } },
  fr: { home: { title: 'Bienvenue' } },
  es: { home: { title: 'Bienvenido' } },
};
```

3. Images (`src/config/images.ts`)

Critical: All image paths must be URL-encoded (spaces → `%20`). Vercel's Linux servers are case-sensitive.

ts

```
// In images.ts:
export const IMAGES = {
  equipment: {
    baler: '/Images/Equipment/Product%20image_baler.jpg',
  },
};

// In components:
import { IMAGES } from '../config/images';
<img src={IMAGES.equipment.baler} alt="Baler" />
```

API Endpoints (Serverless Functions)

All forms submit to `/api/*` endpoints. These are Vercel serverless functions in the `api/` folder.

Endpoint	Purpose	Sends Email To
`POST /api/contact`	Contact form	info@vdrs.com
`POST /api/quote`	Quote requests	info@vdrs.com
`POST /api/application`	Job applications	HR (AChirca@vdrs.com)
`POST /api/test-center`	Test center booking	info@vdrs.com
`POST /api/newsletter/subscribe`	Newsletter signup	(adds to list)

Rate Limiting: 10 requests per 10 seconds per IP.

Testing API Locally

API functions don't run with `npm run dev`. Options:

1. **Deploy to Vercel** (preview deployments test everything)
2. **Use Vercel CLI**: `vercel dev` (runs functions locally)

Key Patterns to Know

Language-Aware Routing

Routes support language prefixes automatically:

- `/equipment` → English
- `/fr/equipment` → French
- `/es/equipment` → Spanish

Animations with Reduced Motion Support

Always use the animation utilities:

```tsx
```

```tsx
import { getMotionConfig } from '../utils/animations';
import { usePrefersReducedMotion } from '../hooks/usePrefersReducedMotion';

function MyComponent() {
  const prefersReducedMotion = usePrefersReducedMotion();
  const motionConfig = getMotionConfig(prefersReducedMotion);

  return <motion.div {...motionConfig.fadeIn}>Content</motion.div>;
}
```

Form Validation

All forms use Zod schemas:

```tsx
import { z } from 'zod';

const schema = z.object({
  email: z.string().email('Invalid email'),
  name: z.string().min(2, 'Too short'),
});
```

Common Commands

```bash
npm run dev          # Start dev server
npm run build        # Production build
npm run type-check   # Check TypeScript errors
npm run lint         # Run ESLint
npm run preview      # Preview production build locally
```

Deployment

Automatic: Push to `main` branch → Deploys to production
Preview: Create PR → Gets preview URL automatically

Environment Variables (Vercel Dashboard)

Required for API functions to send emails:

```
EMAIL_SERVICE=resend
RESEND_API_KEY=re_xxxxx
FROM_EMAIL=noreply@vdrs.com
CONTACT_EMAIL=info@vdrs.com
QUOTE_EMAIL=info@vdrs.com
```

Gotchas & Tips

Issue	Solution
Images 404 on Vercel	URL-encode spaces: `My Image.jpg` → `My%20Image.jpg`
Translations not showing	Check key exists in ALL 3 languages in translations.ts
TypeScript errors on build	Run `npm run type-check` locally first
API not working locally	Use `vercel dev` or test on preview deployment
Rate limited (429 error)	Wait 10 seconds, it's per-IP

File Quick Reference

Need to...	Look in...
Add/edit a page	`src/pages/`
Change navigation	`src/components/Navbar.tsx`
Add translation	`src/config/translations.ts`
Add image path	`src/config/images.ts`
Modify form API	`api/*.ts`
Change routes	`src/App.tsx`
Edit deployment config	`vercel.json`

Questions?

Developer: Ajith Srikanth
Email: asrikanth@vdrs.com
Repo: https://github.com/AjithVanDyk/stagevdrs.git

Last Updated: January 2025